

Activation Steering for Latent Multi-Agent Reasoning

Judger token-efficiency, localization, and what failed upstream

Sagnik Biswas

July 15, 2026

Residual SEAL · native contrastive vectors · KV-cache steering · K-budget CES

One program, four steering experiments

- 1 **SEAL on the Judger** — token-efficient latent MAS (*positive*).
- 2 **Per-agent / native vectors** — where does steering help? (*honest negative* upstream).
- 3 **KV-cache steering** — can we steer the latent handoff? (*same lesson*).
- 4 **K-budget CES** — recover accuracy at smaller latent budget K ? (*Gate 1 no-go*).

Converging message

Leverage lives at the **text-emitting Judger**; upstream latent steering does not move the accuracy–compute frontier.

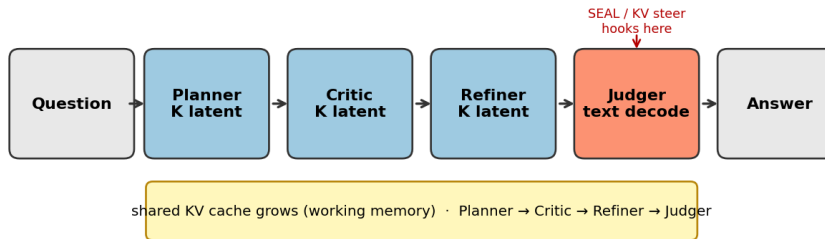
Technical Background: LatentMAS

Paper: Zou et al., *Latent Collaboration in Multi-Agent Systems* (arXiv:2511.20639).

- **Problem with TextMAS:** agents decode $\rightarrow$ re-encode natural language. Lossy (hidden states collapse to tokens) and expensive (full AR decode per message).
- **Latent thought:** instead of sampling a token, take last-layer h_t , (optionally) realign with $W_a \approx W_{\text{out}}^\dagger W_{\text{in}}$, feed as next inputs_embeds. Repeat for K steps — no softmax.
- **Working-memory transfer:** after K steps, pass the full layer-wise **KV cache** to the next agent (prepend). Message = keys/values, not text.
- **Only the Judge decodes text.** Upstream agents emit *zero* tokens by construction.

Architecture at a Glance

LatentMAS sequential pipeline (shared KV; only Judger emits text)



- Sequential topology used throughout: Planner → Critic → Refiner → Judger.
- Default $K=40$ latent forwards per upstream agent; total latent compute $\approx 3(K+1)$.
- Output-token metric = Judger generations to EOS (the only place tokens exist to shrink).

Technical Background: Activation Steering / SEAL

Paper: Chen et al., *SEAL* (arXiv:2504.07986) — training-free reasoning calibration.

- Long CoT mixes **execution** (carry solution forward), **reflection** (“wait / check”), **transition** (“alternatively”). Excess reflection/transition $\Leftrightarrow$ overthinking.
- Thought types are **linearly separable** in deep residual streams.
- Offline, from ~ 50 GSM8K-train traces at layer $L=28$:

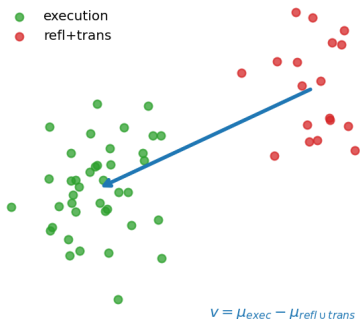
$$v = \mu_{\text{exec}} - \mu_{\text{refl} \cup \text{trans}} \quad (\text{raw } \|v\| \approx 52.7)$$

- Online hook on decoder layer L : $h_{:, -1, :} += \alpha \hat{v}$.
 $\alpha > 0$ suppresses reflection $\Rightarrow$ shorter decode; $\alpha=0$ = plain LatentMAS.

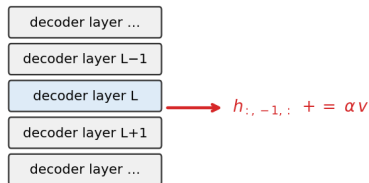
SEAL Mechanically

Offline: contrastive vector at layer L

- execution
- refl+trans



Online: residual-stream hook (last token)



$\alpha > 0$ nudges away from reflection $\Rightarrow$ shorter CoT

- Hook registered once (SealSteerer); enabled/disabled **per agent role** (`--seal_agents`).
- Applied every Judger forward during `generate_text_batch`; negligible latency vs. decode.
- Same residual site later reused for **native** vectors and trainable CES v .

Why Steer the Judger First?

Architectural consequence, not a modeling preference

- Planner / Critic / Refiner run a **fixed** K -step latent loop — no variable-length text.
- Every output token the system emits comes from the Judger's AR decode.
- Therefore a length/token objective can only bite the Judger; upstream SEAL can at best change *quality* of the KV the Judger reads.
- Early pilot: steering latent agents alone $\Rightarrow$ no token change (by construction) and no clear accuracy win.

Research question becomes: does Judger SEAL cut tokens at matched accuracy?

Then: can upstream / handoff / smaller- K steering still help?

Fork Fidelity (before any intervention)

Two independent proofs on Gen-Verse/LatentMAS@9a9e4d3

- **Byte-for-byte:** all 24 upstream file SHAs match (0 mismatches).
- **Behavioral:** Qwen3-14B accuracy within noise of the paper.

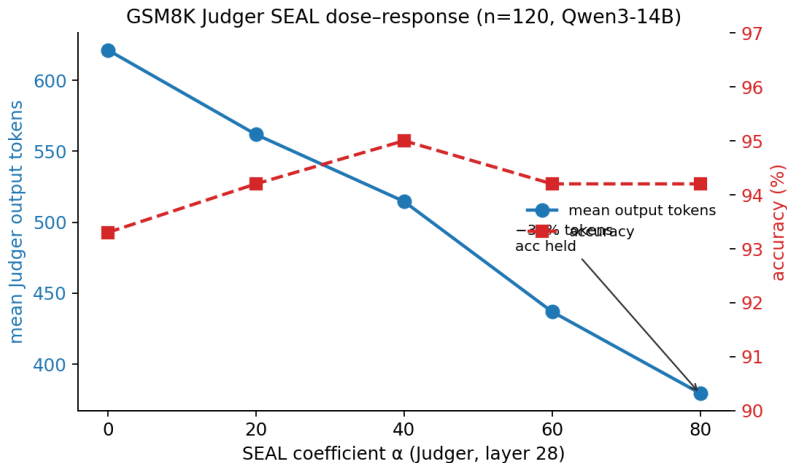
Task	Our fork	Paper
GSM8K	92.7%	95.2%
ARC-Challenge	94.7%	95.6%
MedQA	79.3%	80.7%

Subset / single seed / stochastic decode vs. paper's 3-seed full-set means.

Protocol (SEAL A/B)

Parameter	Value
Model	Qwen/Qwen3-14B (40 layers, $d=5120$)
Topology	sequential; $K=40$ upstream
Datasets	GSM8K, ARC-C, MedQA
n / seed	120 / 100; seed 42
Sampling	$T=0.6$, top- $p=0.95$
SEAL layer / vector	28; GSM8K-train MoD
Metric	accuracy + mean Judger tokens
Backend	HF, $1\times$ H200

Main Result: Judger Dose–Response



- Monotonic token cut: -17% (coef 40) $\rightarrow -39\%$ (coef 80); accuracy held or improved.
- $\sim 2\times$ faster Judger decode at coef 80 — real over-reasoning (~ 620 tok baseline).

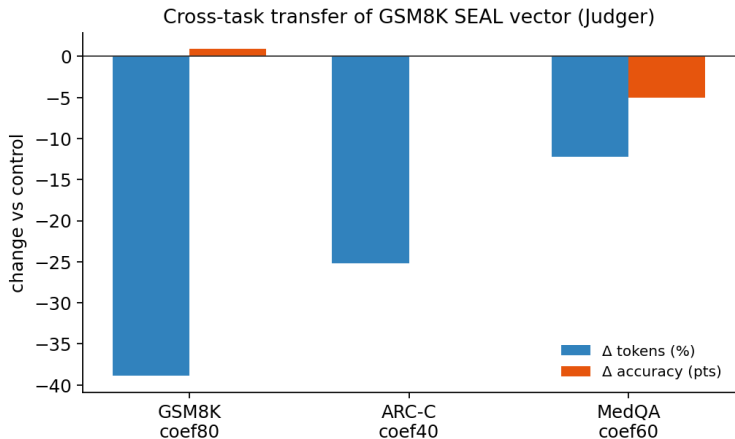
Numbers Behind the Curve

coef α	Accuracy	Mean tokens	Δ tokens
0 (control)	93.3%	621	—
40	95.0%	515	−17%
60	94.2%	437	−30%
80	94.2%	379	− 39%

Interpretation

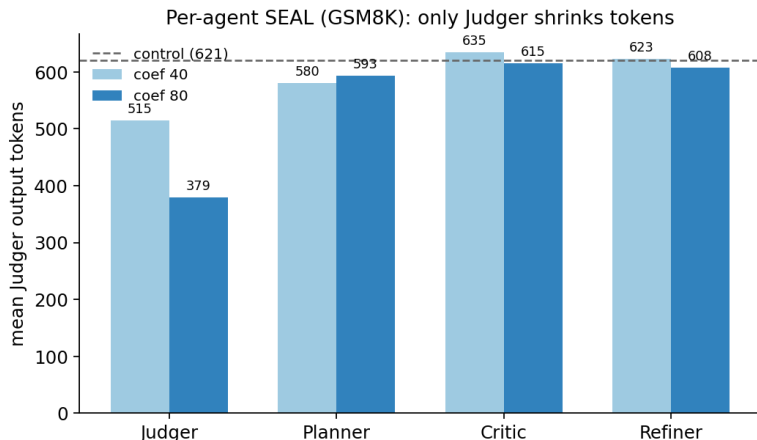
SEAL's execution—reflection axis is a **verbosity / overthinking** direction in Judger residual space. Positive α shortens CoT without collapsing answer quality on GSM8K.

Cross-Task Transfer



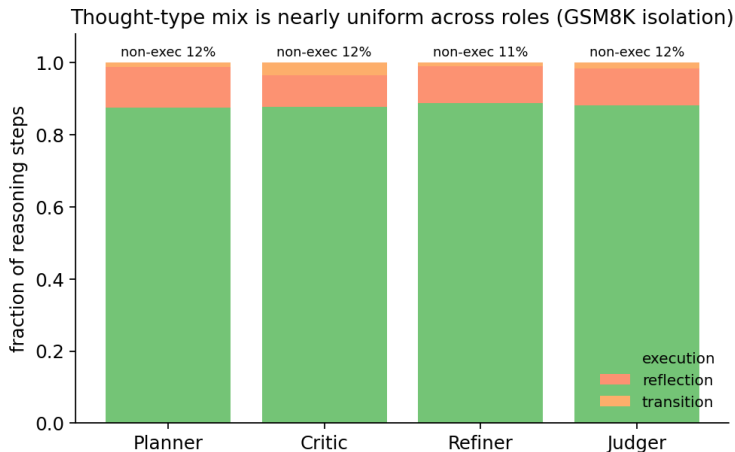
- ARC-C: **−25% tokens at identical accuracy** (coef 40) — clean transfer.
- MedQA: coef-sensitive (early coef60: −12% tok / −5 pts; later held-out coef80: +2 pts / −26%).
- Same GSM8K vector, no re-extraction — supports SEAL's claim of a general overthinking axis.

Per-Agent Ablation: Only Judger Moves Tokens



- Same GSM8K SEAL vector, one role enabled at a time.
- Steering all agents $\approx$ Judger-only — upstream contribution to length is ≈ 0 .

Why? Thought Mix Is Uniform



- ~88% execution for every role — Judger is not “more reflective.”
- Effect is **structural**: only Judger has a variable-length text channel.
- Caveat: isolation proxy (roles prompted alone in text).

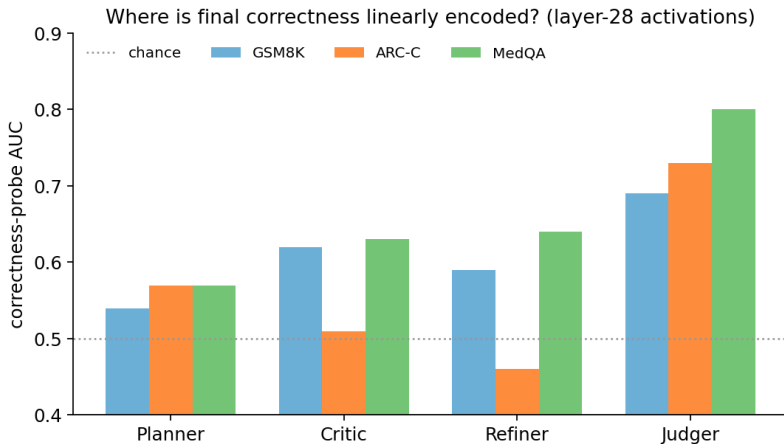
New objective for latent agents (quality, not length)

- Capture each agent's **in-pipeline** layer-28 residual (conditioned on upstream KV), mean-pool per run.
- Label by **final answer correctness**; build

$$v_{\text{agent}} = \mu(h \mid \text{correct}) - \mu(h \mid \text{incorrect}).$$

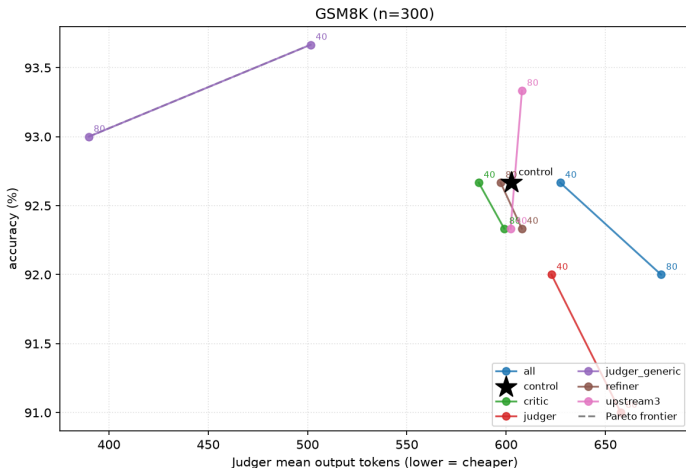
- Apply during that agent's forwards; evaluate end-to-end accuracy + Judger tokens.
- Mechanistic check: 5-fold linear probe AUC of $h \rightarrow \text{correctness}$ (how early is the outcome decided?).

Mechanism: Correctness Is Decided Late



- Upstream AUC 0.46–0.64 (near chance); Judger 0.69–0.80.
- Wrong answers over-generate **+70–82%** tokens — length coupled to failure *at Judger*.
- Predicts: steering fixed-runtime upstream agents cannot smooth a late decision.

Native Sweep: Honest Negative



- Purple **judger_generic** dominates up-and-left; every native/upstream config hugs control.
- Native Judger vector is *worse* than generic SEAL — toward-correct $\neq$ shorter.
- Same story on ARC / MedQA Pareto plots (assets).

Native vs Generic (GSM8K test, $n=300$)

Steering	coef	Acc	Tokens
control	0	92.7%	602
Critic (native)	40	92.7%	586
Upstream-3 (native)	80	93.3%	608
Judger (native)	80	91.0%	658
Judger (generic SEAL)	80	93.0%	390 (−35%)

Verdict

For token efficiency in sequential latent MAS, **Judger-only SEAL is sufficient and dominant**. Isolation-proxy Phase-C critic gains do not survive native end-to-end eval.

Paper: *KV Cache Steering for Controlling Frozen LLMs*
(arXiv:2507.08799).

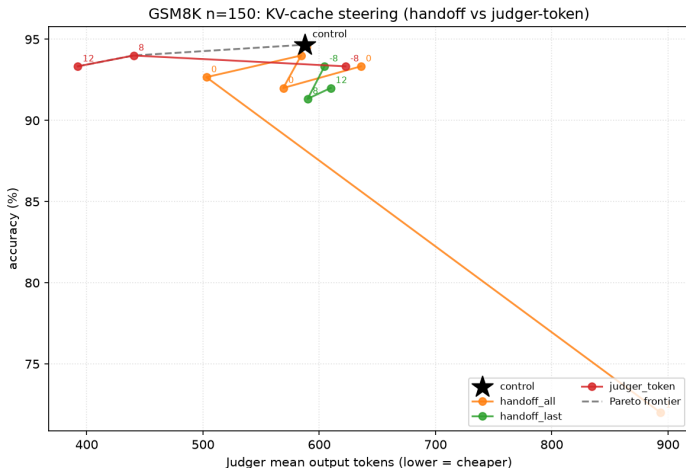
- One-shot edit of past keys/values, then normal decode — *no* per-step residual hook:

$$K += c_k S_k, \quad V += c_v S_v$$

(we use value-only, $c_k=0$).

- Direction S from 200 GSM8K-train contrastive pairs: CoT-rich vs answer-only ICL.
- Surfaces in LatentMAS:
 - `handoff_last` / `handoff_all` — latent KV the Judger inherits;
 - `judger_token` — Judger's own last prompt token (paper's intended surface).

KV Results: Handoff Fails; Judger Token Wins



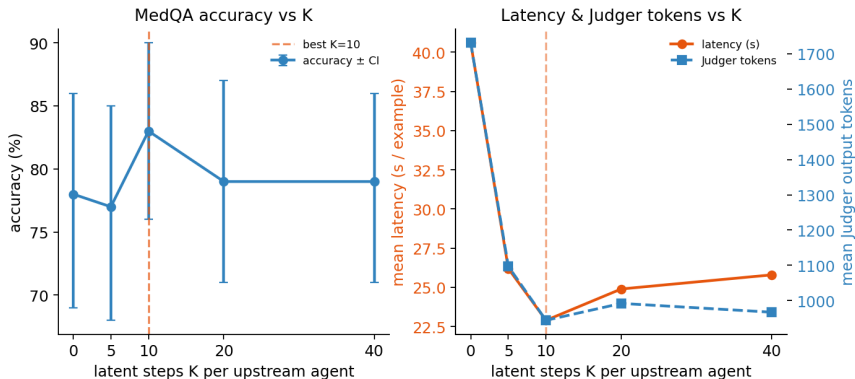
- handoff_all $c_v=1$: accuracy collapses 94.7%→72% (attention saturation — full shift of Judger attention output).
- judger_token $c_v=+12$: **-33% tokens**, accuracy within noise — third independent Judger-side win.

Option 1 claim (now stopped at Gate 1)

- Reduce per-agent latent budget $K_{\text{small}} < K_{\text{full}}$ to save compute.
- Learn a residual steering vector v (CES / ASC-style) applied during upstream **latent steps only**, so small- K recovers full- K accuracy.
- Primary compute metric: **wall-clock latency** (not raw forward count — KV length grows, forwards unequal cost).
- Gate 1 must show: credible accuracy gap *and* $\geq 15\%$ latency savings at K_{small} .
- Gate 2 (grad smoke on Qwen3-4B) already passed; do **not** scale training without Gate 1.

Gate 1: MedQA K-Curve

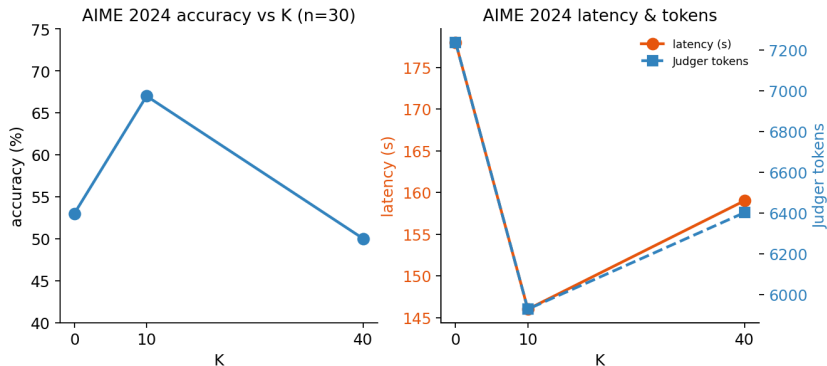
Gate 1: MedQA n=100 @ max_new_tokens=4096 — smaller K is slower



- Best accuracy *and* best latency at $K=10$, not $K=40$.
- Cutting K increases Judger tokens $\Rightarrow$ system gets **slower**.

Gate 1 Probe: AIME 2024 @ 8192

Gate 1 probe @ 8192: $K=10$ best; $K=0$ slower despite fewer latent forwards



- Credible paired gap $K=10 - K=0 \approx +13$ pts — but $K=0$ is $\sim 21\%$ slower.
- Extra latent steps often **shorten** Judge CoT enough to more than pay for themselves.

Gate 1 Verdict: STOP K-Budget CES

Criterion	MedQA	AIME@8192
Credible acc($K_{\text{full}} > K_{\text{small}}$)	Weak / wrong dir.	Yes (10 vs 0)
$\geq 15\%$ wall-clock cut at K_{small}	No	No
Proceed to CES train	No	No

Root cause

Wall-clock is **Judger-token-dominated**. Smaller $K \Rightarrow$ longer Judger CoT $\Rightarrow$ slower system.

Do **not** start CES pair mining for K-budget recovery.

Where Reasoning Lives

Converging evidence: intervention leverage at the Judger



purple = Judger-side win · peach = upstream / handoff null

Four independent lines: residual SEAL, native contrastive, KV surfaces,
and the *K*-curve
all localize effective intervention at the **Judger**.

What's the Contribution

- 1 **Analysis / localization (strongest):** where reasoning is decided in latent MAS, and why upstream steering fails (probes + three methods + K -curve).
- 2 **Method:** Judger residual / cache steering for token-efficient latent MAS (-25 to -39% tokens, accuracy held, transfers).

Preferred pivot after Gate 1

Double down on **Judger-side** ASC / SEAL-style CoT compression — not K -budget recovery.

Limitations

- ① **Power:** $n=100-300$ (AIME $n=30$), mostly single seed.
- ② **Coverage:** one layer (28), one backbone (Qwen3-14B).
- ③ **Task sensitivity:** MedQA coef-dependent.
- ④ **Proxies:** isolation thought-types; capture mean-pools; class imbalance on easy tasks.
- ⑤ **Modality:** SEAL/KV directions extracted from text CoT, applied under latent conditioning.

Next Steps

- $n \approx 500$, 3 seeds, full Judger accuracy–token Pareto.
- Gentler coef / layer sweep for MedQA Judger steering.
- Optional method paper: Judger ASC / learned compression.
- If upstream again: hierarchical topology or process objectives — **not** K-budget wall-clock recovery.
- Position against cache-steering / RepE baselines (partially done).

Core Message

SEAL on the LatentMAS **Judger** cuts output tokens **~25–39% with accuracy retained**, and the vector **transfers**.

The gain is **localized to the text-emitting agent**. Upstream residual, native, and KV handoff steering do not move the frontier; the *K*-curve shows why wall-clock is Judger-dominated.

Stop K-budget CES. Pivot to Judger-side token compression — where steering actually works.